Q38 · MINIMUM FALLING PATH SUM

Step-by-Step Screenshots: Watching the DP Table Fill In

Vanjinathan S · Register No. 192425231 — captured from the animated explainer website, start to finish.

The problem

Given a square matrix, find the minimum sum of a path that falls from the top row to the bottom row, moving one row down at a time — straight down, or diagonally left/right — at each step.

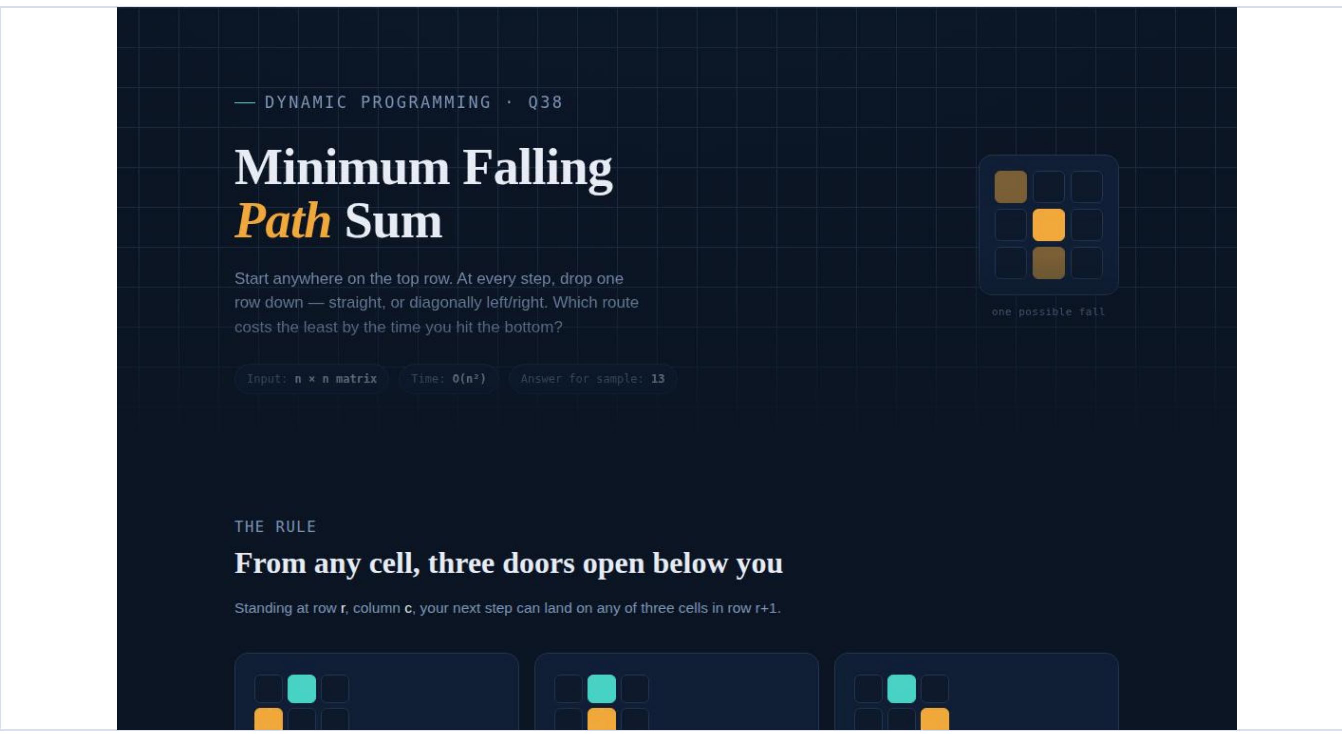
```
matrix = [[2,1,3],[6,5,4],[7,8,9]]  →  Min Sum = 13
```

How it's solved

A second table, dp, is built one row at a time. Each cell dp[r][c] stores the cheapest way to reach that position from the top: dp[r][c] = matrix[r][c] + min(the up-to-three cells directly above it). The smallest value in the final row is the answer, and tracing back through each cell's cheapest parent reveals the actual path taken.

Overview page

The top of the website, showing the problem statement and the rules of movement before scrolling to the interactive demo.

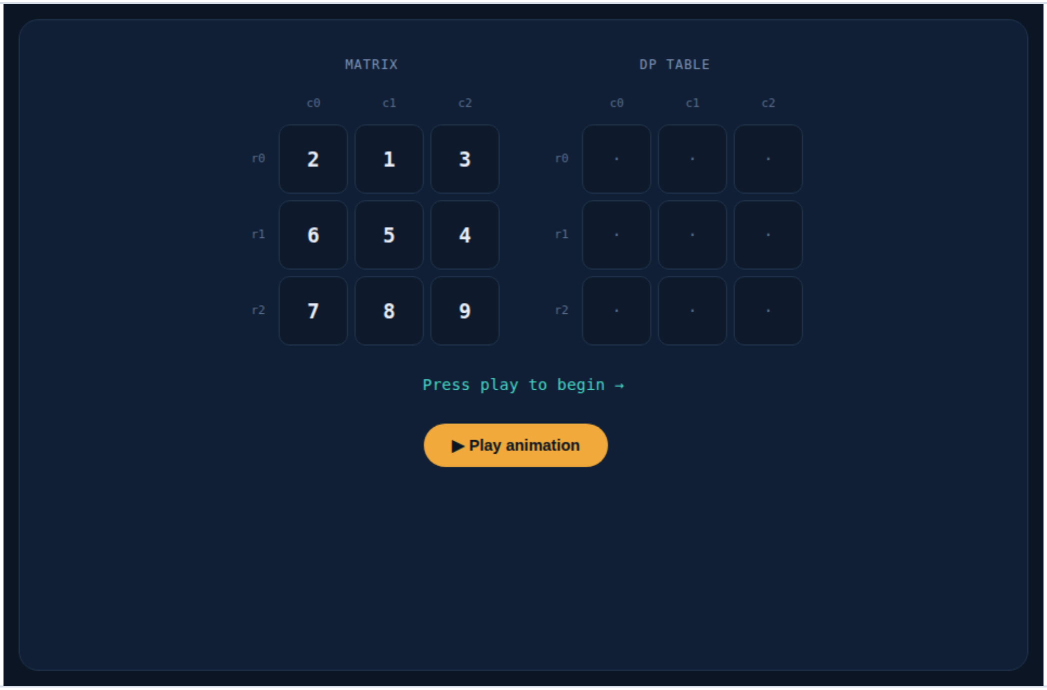


Screenshots: Start to Finish

Each screenshot below was captured while the animation ran, in the order it actually plays out on screen.

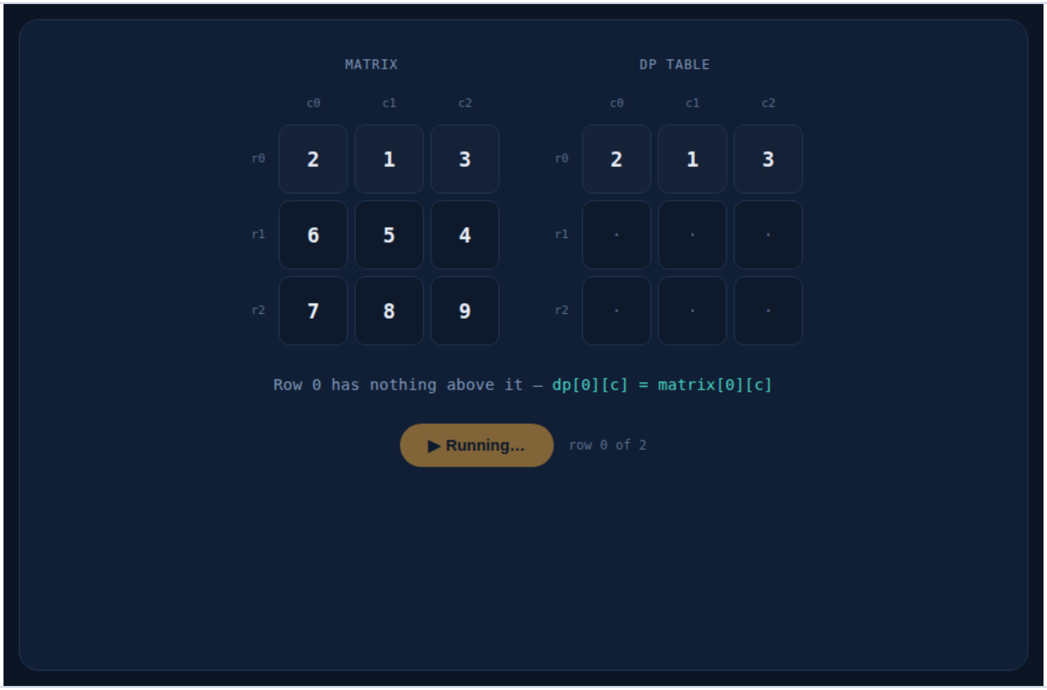
STEP 1 — Starting state — nothing computed yet

The Matrix grid on the left shows the raw input values from the question. The DP table on the right is still empty (each cell shows a dot). Nothing has been calculated yet — this is the screen right before pressing Play.



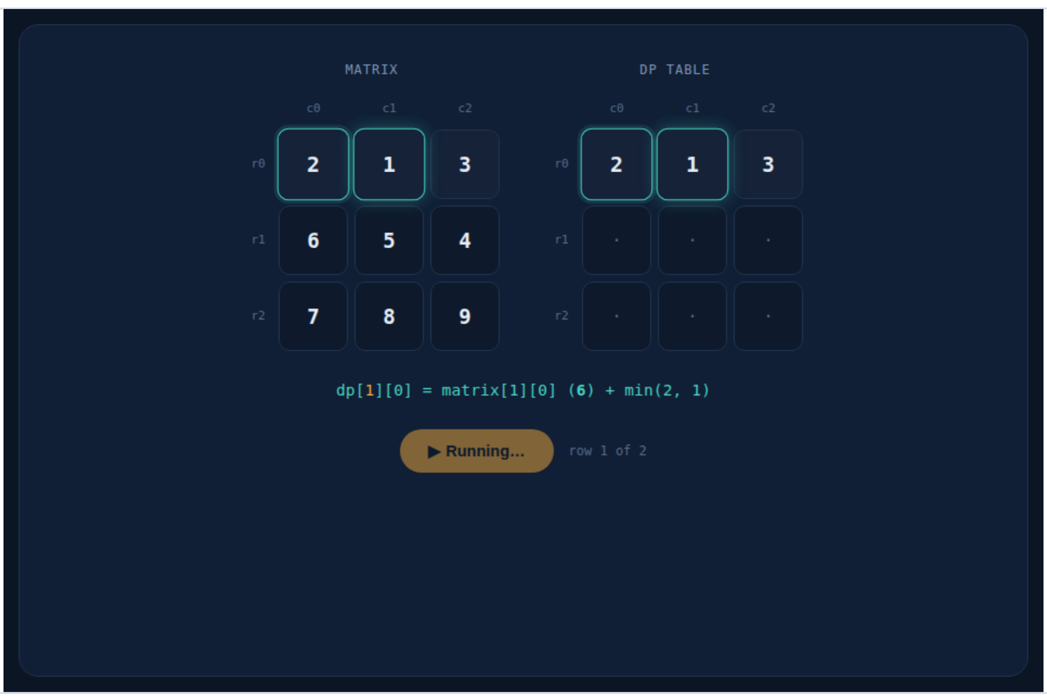
STEP 2 — Row 0 copied across

Row 0 has nothing above it, so there is no choice to make — dp[0][c] is simply set equal to matrix[0][c]. The DP table's top row now reads 2, 1, 3, matching the input exactly.



STEP 3 — Row 1, column 0 — checking the candidates above

Now the real work starts. To fill dp[1][0], the app looks at the cells diagonally/directly above it in row 0 — here, only dp[0][0]=2 and dp[0][1]=1 are valid (there is no column −1). Both are outlined in teal while the app compares them.

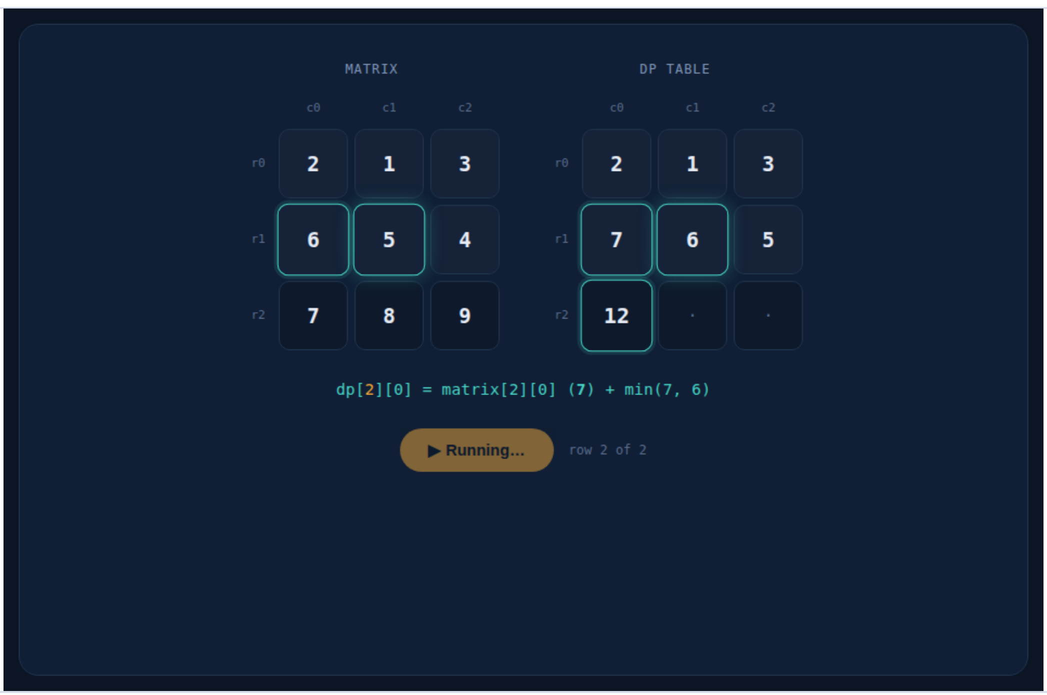


STEP 4 — Row 1, column 1 — three candidates this time

For dp[1][1], all three cells above (dp[0][0]=2, dp[0][1]=1, dp[0][2]=3) are valid parents. The formula bar spells it out: dp[1][1] = matrix[1][1] (5) + min(2, 1, 3) = 5 + 1 = 6. Column 0 of row 1 is already filled in with 7 from the previous step.

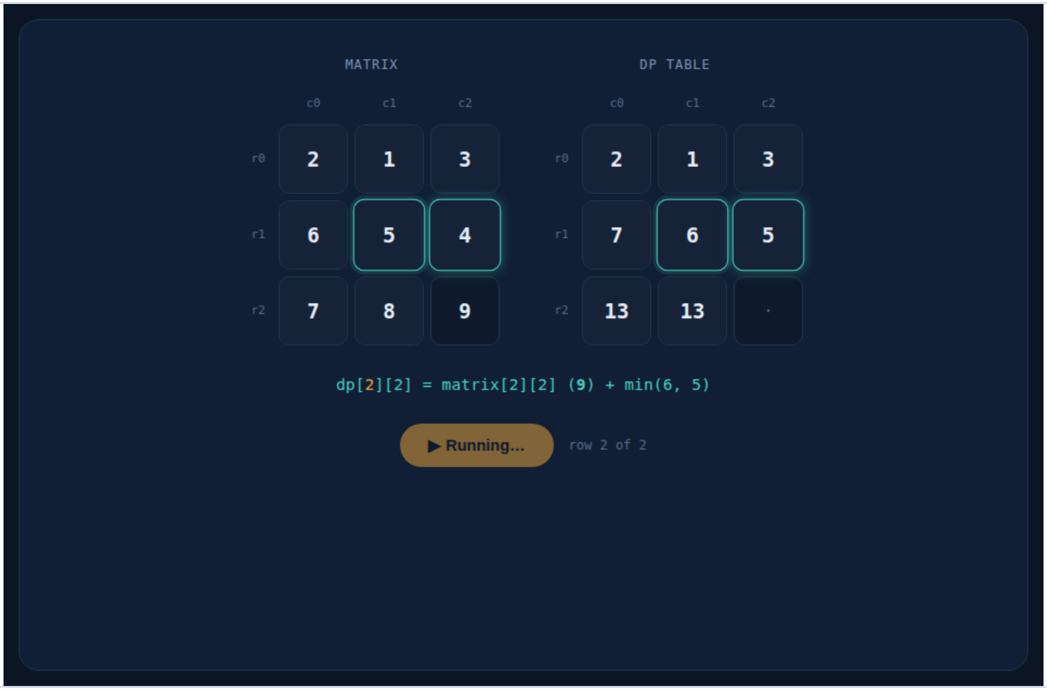
STEP 5 — Row 2, column 0 — the count-up animation

Moving to the last row. While dp[2][0] is being calculated, the number animates upward (caught here mid-count at 12, on its way to the true value of 13 = matrix[2][0] (7) + min(dp[1][0]=7, dp[1][1]=6)). This little detail makes it clear a fresh value is being written, not just appearing instantly.



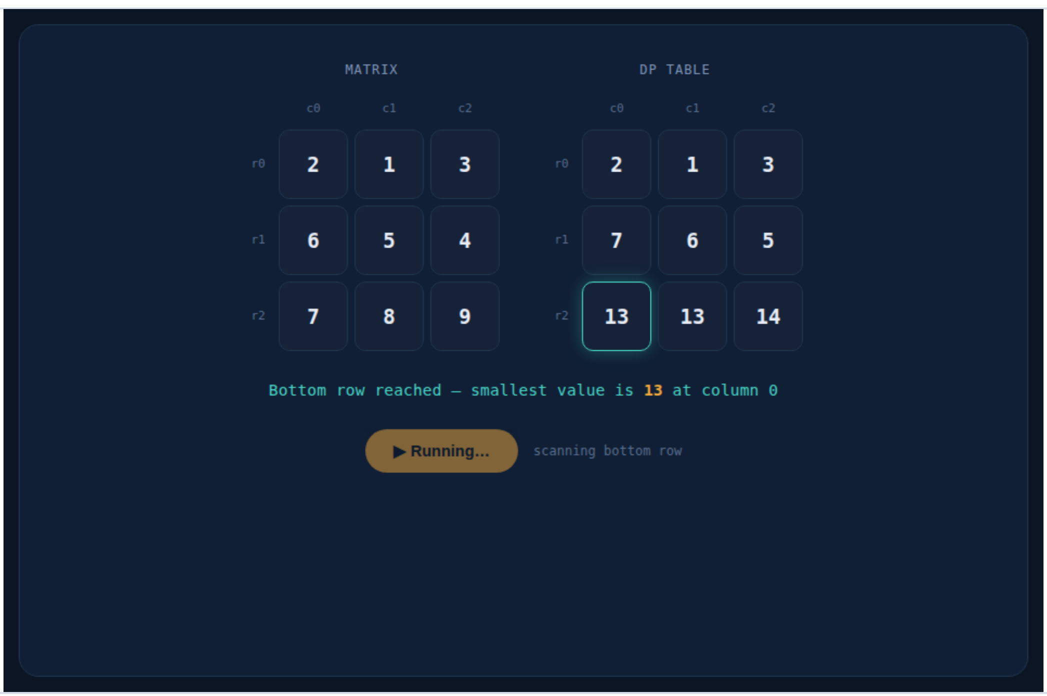
STEP 6 — Row 2, column 2 — the last cell of the table

dp[2][2] = matrix[2][2] (9) + min(dp[1][1]=6, dp[1][2]=5) = 9 + 5 = 14. With this, every cell of the DP table is filled: 13, 13, 14 along the bottom row.



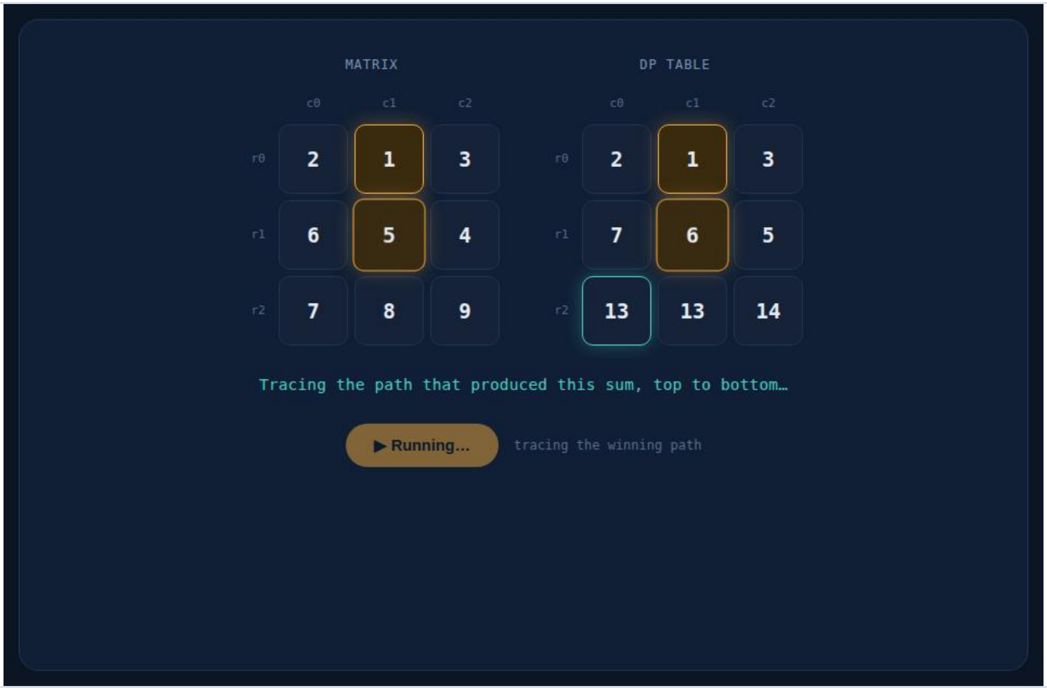
STEP 7 — Scanning the bottom row for the answer

With the table complete, the app scans the last row (13, 13, 14) for the smallest value. It finds 13 at column 0 and highlights that cell — this number is the final answer, but the walk that produced it hasn't been shown yet.



STEP 8 — Walking back up the winning path

Starting from the winning cell, the app walks backward row by row, lighting up in gold exactly which parent produced each value: 13 at (row 2, col 0) came from 6 at (row 1, col 1), which came from 1 at (row 0, col 1). This step is caught mid-trace, with the top two cells of the path already gold.



STEP 9 — Finished — Min Sum = 13

The complete path is highlighted top to bottom in both grids: 1 → 6 → 13 in the DP table, matching 1 → 5 → 7 in the original matrix. The result panel below confirms it plainly: Path 1 → 5 → 7 = 1 + 5 + 7 = 13, exactly the expected output from the question.

Result

Min Sum = 13, following the path 1 → 5 → 7 through the matrix — matching the expected output given in the question exactly.